Cif2x Documentation

リリース *1.1.0*

ISSP, University of Tokyo

2026 年 10 月 30 日

目次

第 1 章　概要 1
1.1 cif2x とは? . . . 1
1.2 ライセンス . . . 1
1.3 開発貢献者 . . . 1
1.4 バージョン履歴 . . . 2
1.5 コピーライト . . . 2
1.6 動作環境 . . . 2

第 2 章　インストールと基本的な使い方 3

第 3 章　チュートリアル 7
3.1 入力パラメータファイルを作成する . . . 7
3.2 第一原理計算入力ファイルを生成する . . . 9
3.3 パラメータセットを指定する . . . 11
3.4 RESPACK ワークフロー . . . 12

第 4 章　コマンドリファレンス 19
4.1 cif2x . . . 19

第 5 章　ファイルフォーマット 22
5.1 入力パラメータファイル . . . 22
5.2 Quantum ESPRESSO 向けパラメータ . . . 24
5.3 VASP 向けパラメータ . . . 26
5.4 OpenMX 向けパラメータ . . . 28
5.5 AkaiKKR 向けパラメータ . . . 28

第 6 章　拡張ガイド 29
6.1 Quantum ESPRESSO の mode を追加する . . . 29
6.2 対応する計算モード . . . 30
6.3 トラブルシューティング . . . 31

第 7 章　CIF データ取得ツール (getcif) 33
7.1 概要 . . . 33
7.2 チュートリアル . . . 33

7.3 コマンドリファレンス 37
7.4 ファイルフォーマット 37
7.5 パラメータリスト 40

第 1 章

概要

1.1 cif2x とは?

近年、機械学習を活用した物性予測や物質設計 (マテリアルズインフォマティクス) が注目されています。機械学習の精度は、適切な教師データの準備に大きく依存しています。そのため、迅速に教師データを生成するためのツールや環境の整備は、マテリアルズインフォマティクスの研究進展に大きく貢献すると期待されます。

cif2x は、cif ファイルから第一原理計算用の入力ファイルを生成するツールです。入力パラメータを雛形として、物質の種類や計算条件によって変わる箇所を結晶構造データなどから構成します。特定の計算条件に応じた複数の入力ファイルを生成することが可能です。現在は、VASP, Quantum ESPRESSO, OpenMX, AkaiKKR に対応しています。

付属プログラムとして、物質材料データベースから結晶構造データ等を取得するツール getcif を用意しています。現在は Materials Project からのデータ取得に対応しています。物質の組成や対称性、バンドギャップなどの物性値をもとにデータベースを検索し、データを取得することができます。

1.2 ライセンス

本ソフトウェアのプログラムパッケージおよびソースコード一式は GNU General Public License version 3 (GPL v3) に準じて配布されています。

1.3 開発貢献者

本ソフトウェアは以下の開発貢献者により開発されています。

- 開発者
  - 吉見 一慶 (東京大学 物性研究所)
  - 青山 龍美 (東京大学 物性研究所)
  - 本山 裕一 (東京大学 物性研究所)

  - 福田 将大 (東京大学 物性研究所)
  - 井戸 康太 (東京大学 物性研究所)
  - 福島 鉄也 (産業技術総合研究所)
  - 笠松 秀輔 (山形大学 学術研究院 (理学部主担当))
  - 是常 隆　(東北大学大学院理学研究科)
- プロジェクトコーディネーター
  - 尾崎 泰助 (東京大学 物性研究所)

1.4 バージョン履歴

ver.1.1.0
: 2024/09/14 リリース

ver.1.0.1
: 2024/03/31 リリース

ver.1.0.0
: 2024/03/19 リリース

ver.1.0-alpha
: 2023/12/28 リリース

1.5 コピーライト

© 2023- The University of Tokyo. All rights reserved.

本ソフトウェアは 2023 年度 東京大学物性研究所 ソフトウェア高度化プロジェクトの支援を受け開発されており、その著作権は東京大学が所持しています。

1.6 動作環境

以下の環境で動作することを確認しています。

- Ubuntu Linux + python3

第 2 章

インストールと基本的な使い方

必要なライブラリ・環境

HTP-tools に含まれる第一原理計算入力ファイル生成ツール cif2x および CIF データ取得ツール getcif を利用するには、以下のプログラムとライブラリが必要です。(これらは `pyproject.toml` に記載されており、インストール時に自動的に導入されます。)

- Python 3.11 以上
- numpy モジュール
- pandas モジュール
- pymatgen モジュール
- f90nml モジュール
- beautifulsoup4 モジュール
- mp_api モジュール
- phonopy モジュール
- ruamel.yaml モジュール

AkaiKKR 向けの入力ファイルを生成する場合は、上記に加えて AkaiKKRPythonUtil モジュールが必要です。これは自動ではインストールされないため、別途インストールします (下記参照)。

ソースコード配布サイト

- GitHub リポジトリ

ダウンロード方法

git を利用できる場合は、以下のコマンドで cif2x をダウンロードできます。

```
$ git clone https://github.com/issp-center-dev/cif2x.git
```

インストール方法

cif2x をダウンロード後、以下のコマンドを実行してインストールします。cif2x が利用するライブラリも必要に応じてインストールされます。

```
$ cd ./cif2x
$ python3 -m pip install .
```

実行プログラム `cif2x`, `getcif` がインストールされます。

なお、AkaiKKRPythonUtil モジュールは別途インストールが必要です。以下の手順で 配布サイト からソースコードを取得しインストールします。また、必要な seaborn モジュールもインストールしておきます。

```
$ git clone https://github.com/AkaiKKRteam/AkaiKKRPythonUtil.git
$ cd ./AkaiKKRPythonUtil/library/PyAkaiKKR
$ python3 -m pip install .
$ python3 -m pip install seaborn
```

ディレクトリ構成

```
.
|-- LICENSE
|-- README.md
|-- pyproject.toml
|-- docs/
|   |-- ja/
|   |-- tutorial/
|-- src/
|   |-- cif2x/
|       |-- __init__.py
|       |-- main.py
|       |-- cif2struct.py
|       |-- struct2qe.py
|       |-- qe/
|       |   |-- __init__.py
|       |   |-- calc_mode.py
|       |   |-- cards.py
|       |   |-- content.py
|       |   |-- qeutils.py
```

(次のページに続く)

(前のページからの続き)

```
|       |   |-- tools.py
|       |-- struct2vasp.py
|       |-- struct2openmx.py
|       |-- openmx/
|       |   |-- __init__.py
|       |   |-- vps_table.py
|       |-- struct2akaikkr.py
|       |-- akaikkr/
|       |   |-- make_input.py
|       |   |-- read_input.py
|       |   |-- run_cif2kkr.py
|       |-- utils.py
|   |-- getcif/
|       |-- __init__.py
|       |-- main.py
|-- sample/
```

基本的な使用方法

cif2x は第一原理計算プログラムのための入力ファイルを生成するツールです。入力パラメータを雛形として、物質の種類や計算条件によって変わる箇所を結晶構造データなどから構成します。現在は Quantum ESPRESSO, VASP, OpenMX, および AkaiKKR の入力ファイル形式に対応しています。

1. 入力パラメータファイルの作成

   cif2x を使用するには、まず、生成する入力ファイルの内容を記述したパラメータファイルを YAML 形式で作成します。詳細についてはファイルフォーマットの章を参照してください。

2. 結晶構造ファイルと擬ポテンシャルファイルの配置

   対象となる物質の結晶構造を記述したファイルを用意します。ファイル形式は CIF または pymatgen が扱える POSCAR や xsf 形式に対応しています。

   Quantum ESPRESSO の場合、利用する擬ポテンシャルファイルと、CSV 形式のインデックスファイルを配置します。擬ポテンシャルファイルの配置先などは入力パラメータファイル内に指定します。

   VASP の場合、擬ポテンシャルファイルの格納場所を `~/.config/.pmgrc.yaml` ファイルに記述するか環境変数にセットします。入力パラメータファイル内で指定することもできます。

3. コマンドの実行

   作成した入力パラメータファイルおよび結晶構造データファイルを入力として cif2x プログラムを実行します。Quantum ESPRESSO 用の入力ファイルを生成する場合はターゲットオプションに

-t QE を指定します。VASP の場合は -t VASP, OpenMX の場合は -t OpenMX, AkaiKKR の場合は -t akaikkr を指定します。

```
$ cif2x -t QE input.yaml material.cif
```

第 3 章

チュートリアル

第一原理計算入力ファイル生成ツール cif2x を使うには、入力パラメータファイルと結晶構造データおよび擬ポテンシャルファイルを用意した後、プログラム cif2x を実行します。現在は Quantum ESPRESSO, VASP, OpenMX, AkaiKKR の入力ファイル生成に対応しています。以下では、 `docs/tutorial/cif2x` ディレクトリにある Quantum ESPRESSO 向けサンプルを例にチュートリアルを実施します。

3.1 入力パラメータファイルを作成する

入力パラメータファイルには、第一原理計算プログラムに与える入力ファイルの内容を記述します。

以下に入力パラメータファイルのサンプルを記載します。このファイルは YAML 形式のテキストファイルで、結晶構造データに対するオプションの指定や、出力する第一原理計算入力ファイルの内容を記述します。仕様の詳細については **ファイルフォーマット** の章を参照してください。

YAML フォーマットでは、 `keyword: value` の辞書形式でパラメータを記述します。 `value` には数値や文字列などのスカラー値や、複数の値を `[ ]` または箇条書きの形式で列挙するリスト型、または辞書型を入れ子にすることができます。

```yaml
structure:
  use_ibrav: false
  tolerance: 0.05

optional:
  pseudo_dir: ./pseudo
  pp_file: ./pseudo/pp_psl_pbe_rrkjus.csv

tasks:
  - mode: scf
    output_file: scf.in
    output_dir: scf
```

(次のページに続く)

(前のページからの続き)

```
  template: scf.in_tmpl
  content:
    namelist:
      control:
        prefix: pwscf
        pseudo_dir:
        outdir: ./work
      system:
        ecutwfc:
        ecutrho:
    CELL_PARAMETERS:
    ATOMIC_SPECIES:
    ATOMIC_POSITIONS:
      option: crystal
    K_POINTS:
      option: automatic
      grid: [8,8,8]
```

入力パラメータファイルは structure, optional, tasks のブロックから構成されます。structure は結晶構造データに関するオプションを指定します。optional は擬ポテンシャルに関する global な設定などを行います。

tasks は出力する第一原理計算入力ファイルの内容を指定します。一連の計算に対応して複数のファイルを生成できるよう、tasks は配列の値を取ります。各出力について、計算内容は mode パラメータで指定します。SCF 計算の scf や NSCF 計算の nscf に対応するほか、一般的な出力を行う任意の出力モードを指定できます。ファイルは output_dir および output_file で指定するファイルに書き出されます。

出力内容は content に記載します。Quantum ESPRESSO の入力ファイルは、 &keyword で始まる Fortran90 の namelist 形式と、 K_POINTS などのキーワードで始まり空行で分割される cards と呼ばれるブロックからなります。content には namelist と cards を入れ子の辞書形式で指定します。いくつかの例外を除いて、指定された内容が基本的にはそのまま入力ファイルに書き出されます。値が空欄のキーワードは、結晶構造データなどから求めた値が代入されます。

また、template として入力ファイルの雛形を指定することもできます。 template に指定したファイルの内容と content のデータを合わせたものを入力データとして扱います。同じキーワードのデータが存在する場合は content の指定が優先されます。従って、既存の入力ファイルを元に必要な箇所を入力パラメータファイルで上書きする使い方が可能です。上記の例では次のファイル (scf.in_tmpl) を template として取り込み、カットオフと CELL_PARAMETER, ATOMIC_SPECIES, ATOMIC_POSITIONS, K_POINT の箇所を結晶構造等から決めます。ecutwfc と ecutrho が空欄で上書きされていることに留意してください。

```
&control
    calculation = 'scf'
```

(次のページに続く)

(前のページからの続き)

```
    prefix = 'pwscf'
    pseudo_dir = './pseudo'
    outdir = './work'
    tstress = .true.
    tprnfor = .true.
/

&system
    ibrav = 0
    nat = 7
    ntyp = 3
    ecutwfc = 36.0
    ecutrho = 180.0
    occupations = 'smearing'
    smearing = 'm-p'
    degauss = 0.01
    noncolin = .true.
    nspin = 2
/

&electrons
    missing_beta = 0.1
    conv_thr = 1e-08
/
```

3.2 第一原理計算入力ファイルを生成する

入力パラメータファイル (input.yaml) と結晶構造データ (Co3SnS2_nosym.cif) を入力として cif2x を実行します。

```
$ cif2x -t QE input.yaml Co3SnS2_nosym.cif
```

実行にあたり、擬ポテンシャルに関する以下の準備が必要です。

- **擬ポテンシャルファイル (.UPF) の配置**: 計算に用いる Quantum ESPRESSO の擬ポテンシャルファイル (.UPF 形式) を、入力パラメータファイルの `optional.pseudo_dir` で指定したディレクトリ (この例では `./pseudo`) に配置します。擬ポテンシャルファイル自体はリポジトリには含まれていないため、PSlibrary や Quantum ESPRESSO の公式擬ポテンシャル配布ページなどから、使用する擬ポテンシャルライブラリに対応するファイルを入手してください。
- **インデックスファイル (CSV) の配置**: 元素種と擬ポテンシャル名を対応付けるインデックスファイ

ルを optional.pp_file (この例では ./pseudo/pp_psl_pbe_rrkjus.csv) に指定します。このサンプルで用いる PSlibrary (PBE, RRKJUS) 用のインデックスファイルは docs/tutorial/cif2x/pseudo/pp_psl_pbe_rrkjus.csv として同梱されています。別の擬ポテンシャルライブラリを使う場合は、これにならって作成してください。

なお、 optional.pseudo_dir は cif2x が .UPF ファイルを探してカットオフ等の情報を取得するためのディレクトリで、生成される入力ファイル内の Quantum ESPRESSO の control.pseudo_dir (計算実行時に pw.x が擬ポテンシャルを参照するパス) とは独立です。後者は content の namelist で指定し、空欄にしておくと cif2x が optional.pseudo_dir の値で補完します (この例では ./pseudo が書き込まれます)。

cif2x を実行すると Quantum ESPRESSO 用の入力ファイルが生成され出力されます。出力先は入力パラメータファイル内のパラメータで指定するディレクトリ (output_dir) およびファイル (output_file) です。この例では ./scf/scf.in に SCF 計算用の入力ファイルが書き出されます。生成されたファイルの内容は次のようになります。

```
&control
    calculation = 'scf'
    prefix = 'pwscf'
    pseudo_dir = './pseudo'
    outdir = './work'
    tstress = .true.
    tprnfor = .true.
/

&system
    ibrav = 0
    nat = 7
    ntyp = 3
    ecutwfc = 35.7641030992696
    ecutrho = 179.501065844332
    occupations = 'smearing'
    smearing = 'm-p'
    degauss = 0.01
    nspin = 2
/

&electrons
    missing_beta = 0.1
    conv_thr = 1e-08
/
CELL_PARAMETERS {angstrom}
```

(次のページに続く)

(前のページからの続き)

```
4.666246  0.000000  2.680170
1.551393  4.400799  2.680170
0.000000  0.000000  5.381186

ATOMIC_SPECIES
Co  58.933195  Co.pbe-n-rrkjus_psl.0.2.3.UPF
Sn  118.710000  Sn.pbe-dn-rrkjus_psl.0.2.UPF
S  32.065000  S.pbe-n-rrkjus_psl.0.1.UPF

ATOMIC_POSITIONS {crystal}
Co  0.500000  0.000000  0.000000
Co  0.000000  0.500000  0.000000
Co  0.000000  0.000000  0.500000
Sn  0.500000  0.500000  0.500000
Sn  0.000000  0.000000  0.000000
S  0.719510  0.719510  0.719510
S  0.280490  0.280490  0.280490

K_POINTS {automatic}
8  8  8  0  0  0
```

カットオフ ecutwfc, ecutrho の値が擬ポテンシャルファイルから取得され、 CELL_PARAMETERS, ATOMIC_POSITIONS および ATOMIC_SPECIES の元素種が結晶構造データから決まり、 ATOMIC_SPECIES の擬ポテンシャルファイル名が optional.pp_file の対応付けから決まり、 K_POINTS が content.K_POINTS の設定 (この例では grid: [8,8,8]) から生成されていることが確認できます。

VASP, OpenMX, AkaiKKR 向けのサンプルは、リポジトリの sample/cif2x/ ディレクトリ以下に用意されています。

3.3 パラメータセットを指定する

入力パラメータ内の値をいくつか変えながら一連の入力ファイルを生成したいことがあります。例えばカットオフの値や k 点の数を変えて収束性を評価するなどの場合です。入力パラメータには値のリストや範囲を指定することができ、値の組み合わせごとに個別のディレクトリを作成して入力ファイルを生成します。パラメータセットの指定は特別な構文 ${...} を用います。

```
content:
  K_POINTS:
    option: automatic
    grid:   ${ [ [4,4,4], [8,8,8], [12,12,12] ] }
```

例えば上記のように K_POINTS を指定すると、 grid の値が [4,4,4], [8,8,8], [12,12,12] の入力ファイルがそれぞれ 4x4x4/, 8x8x8/, 12x12x12/ サブディレクトリ内に作成されます。

同じ ${...} 構文は Quantum ESPRESSO だけでなく全ターゲットで利用でき、出力サブディレクトリ名はスイープ値から生成されます。以下の各 content: ブロックは、前述の例と同様に tasks: の各エントリ配下に置かれます。スイープ対象のキーは content 内でそのパラメータが定義される階層に記述します。VASP では incar (または kpoints/poscar/potcar) の下、OpenMX と AkaiKKR では content 直下のフラットなキーです。

VASP(カットオフ収束):

```
content:
  incar:
    ENCUT: ${ [400, 600, 800] }
```

により 400/, 600/, 800/ に入力ファイルが生成されます。

OpenMX(エネルギーカットオフ収束):

```
content:
  scf.energycutoff: ${ [150, 200, 250] }
```

により 150/, 200/, 250/ に入力ファイルが生成されます。

AkaiKKR(ブリルアンゾーン品質の収束):

```
content:
  bzqlty: ${ [12, 16, 20] }
```

により 12/, 16/, 20/ に入力ファイルが生成されます。

3.4 RESPACK ワークフロー

cif2x は、cRPA 計算パッケージ RESPACK に至る一連のワークフロー (Quantum ESPRESSO による SCF/NSCF 計算 → qe2respack による変換 → RESPACK 実行) のための入力ファイルをまとめて生成できます。cif2x の役割は入力ファイルの生成までで、計算の実行そのものは行いません。以下では docs/tutorial/cif2x/respack ディレクトリにある SrVO3 (立方晶ペロブスカイト) のサンプルを例に、3 つの入力ファイル (scf/scf.in, nscf/nscf.in, input.in) を生成します。

3.4.1 入力パラメータファイル

```
structure:
  use_primitive: true
  use_ibrav: false
```

(次のページに続く)

(前のページからの続き)

```
optional:
  pseudo_dir: ./pseudo
  pp_file: pp.csv
  cutoff_file: cutoff.csv

tasks:
  - mode: scf
    output_file: scf.in
    output_dir: scf
    content:
      namelist:
        control:
          prefix: pwscf
          pseudo_dir:
          outdir: ./work
        system:
          ibrav:
          nat:
          ntyp:
          ecutwfc:
          ecutrho:
        electrons:
          conv_thr: 1.e-8
      CELL_PARAMETERS:
      ATOMIC_SPECIES:
      ATOMIC_POSITIONS:
        option: crystal
      K_POINTS:
        option: automatic
        grid: [6, 6, 6]
  - mode: nscf
    output_file: nscf.in
    output_dir: nscf
    content:
      namelist:
        control:
          prefix: pwscf
          pseudo_dir:
```

(次のページに続く)

(前のページからの続き)

```
      outdir: ./work
    system:
      ibrav:
      nat:
      ntyp:
      ecutwfc:
      ecutrho:
      nbnd: 60
      nosym: true
      noinv: true
    electrons:
      conv_thr: 1.e-8
  CELL_PARAMETERS:
  ATOMIC_SPECIES:
  ATOMIC_POSITIONS:
    option: crystal
  K_POINTS:
    option: crystal
    grid: [6, 6, 6]
- mode: respack
  output_file: input.in
  template: respack.in_tmpl
```

tasks には scf, nscf, respack の 3 つのタスクを記述します。scf / nscf タスクは Quantum ESPRESSO 向けと同じ書式 (content.namelist と cards) で記述します。RESPACK 固有の注意点は次のとおりです。

- structure.use_primitive: true と use_ibrav: false は必須です。RESPACK 用の高対称 k 経路 (pymatgen の HighSymmKpath) が標準化された primitive cell を前提とするためです。
- nscf タスクでは system に nosym: true / noinv: true を指定します。qe2respack が対称性で畳まれていない一様 k メッシュを要求するためで、K_POINTS も option: crystal により全 k 点を列挙する形式にします。
- nbnd は Wannier 関数の構成と cRPA 遮蔽計算に必要な空バンド数を確保する物理パラメータで、ユーザーが指定します (この例では 60)。

3.4.2 RESPACK テンプレート

`respack` タスクは、`template` に指定した RESPACK の namelist 雛形と結晶構造から `input.in` を生成します。テンプレートは `&param_*` namelist のみで構成します (namelist の外側に内容を書くことはできません)。

```
&param_chiqw
Ecut_for_eps = 5.0
flg_cRPA = 1
/
&param_wannier
N_wannier = 3
Lower_energy_window = 11.0
Upper_energy_window = 14.2
/
&param_interpolation
dense = 8, 8, 8
/
&param_calc_int
/
```

物理量はユーザーがテンプレートで与えます: `N_wannier = 3` (V の t2g 軌道)、エネルギー窓 `Lower_energy_window` / `Upper_energy_window`、補間 k メッシュ `dense`、および `&param_chiqw` の cRPA 設定 (`flg_cRPA = 1`) です。一方、`&param_interpolation` の高対称 k 経路 (`N_sym_points` と座標ブロック) は cif2x が結晶構造から自動生成し、Wannier 関数の初期推定は SCDM (`N_initial_guess = 0`) に固定されます。

3.4.3 擬ポテンシャルの準備

RESPACK は Kohn-Sham 波動関数を直接扱うため、ノルム保存 (ONCV) 擬ポテンシャルを使用します。この例では SG15 ONCV (PBE, v1.0) を使用します。cif2x は擬ポテンシャルファイル名を **<元素>.<名前>.UPF** の形式で組み立てるため、ダウンロード後にリネームして `optional.pseudo_dir` (この例では `./pseudo`) に配置します。

```
$ mkdir -p pseudo
$ cd pseudo
$ for el in Sr V O; do
>   curl -LO "http://www.quantum-simulation.org/potentials/sg15_oncv/upf/${el}_ONCV_PBE-
↪1.0.upf"
>   mv ${el}_ONCV_PBE-1.0.upf ${el}.ONCV_PBE-1.0.UPF
> done
```

元素種と擬ポテンシャル名の対応付けは `optional.pp_file` (`pp.csv`) で行います。

```
element,pseudopotential,nexclude,orbitals
Sr,ONCV_PBE-1.0,0,
V,ONCV_PBE-1.0,0,
O,ONCV_PBE-1.0,0,
```

ONCV の UPF ヘッダーにはカットオフの推奨値が含まれないため、この例ではカットオフを optional.cutoff_file (cutoff.csv) で与えます (ecutwfc = 80 Ry, ecutrho = 320 Ry。値は目安であり、実際の計算では収束を確認してください)。

```
pseudofile,ecutwfc,ecutrho
Sr.ONCV_PBE-1.0.UPF,80.0,320.0
V.ONCV_PBE-1.0.UPF,80.0,320.0
O.ONCV_PBE-1.0.UPF,80.0,320.0
```

3.4.4 入力ファイルを生成する

```
$ cif2x -t respack input.yaml SrVO3.cif
```

SCF 計算用の入力ファイルが scf/scf.in に書き出されます。

```
! generated by cif2x.py
&control
    prefix = 'pwscf'
    pseudo_dir = './pseudo'
    outdir = './work'
    calculation = 'scf'
/

&system
    ibrav = 0
    nat = 5
    ntyp = 3
    ecutwfc = 80.0
    ecutrho = 320.0
/

&electrons
    conv_thr = 1e-08
/
CELL_PARAMETERS {angstrom}
```

(次のページに続く)

(前のページからの続き)

```
3.842500  0.000000  0.000000
-0.000000  3.842500  0.000000
0.000000  0.000000  3.842500

ATOMIC_SPECIES
Sr  87.620000  Sr.ONCV_PBE-1.0.UPF
V  50.941500  V.ONCV_PBE-1.0.UPF
O  15.999400  O.ONCV_PBE-1.0.UPF

ATOMIC_POSITIONS {crystal}
Sr  0.000000  0.000000  0.000000
V  0.500000  0.500000  0.500000
O  0.500000  0.500000  0.000000
O  0.500000  0.000000  0.500000
O  0.000000  0.500000  0.500000

K_POINTS {automatic}
6  6  6  0  0  0
```

NSCF 計算用の入力ファイル nscf/nscf.in では、calculation = 'nscf' に加えて nosym, noinv, nbnd が設定され、K_POINTS crystal に全 k 点が列挙されます (紙面の都合で k 点リストは省略します)。

RESPACK 用の制御ファイルは input.in に書き出されます。&param_interpolation の namelist 終端 (/) の直後に、高対称 k 経路の座標ブロックが自動生成されている点に注意してください。

```
&param_chiqw
    ecut_for_eps = 5.0
    flg_crpa = 1
/
&param_wannier
    lower_energy_window = 11.0
    n_initial_guess = 0
    n_wannier = 3
    upper_energy_window = 14.2
/
&param_interpolation
    dense = 8, 8, 8
    n_sym_points = 6, 2
/
0.000000 0.000000 0.000000  ! \Gamma
```

(次のページに続く)

(前のページからの続き)

```
0.000000 0.500000 0.000000  ! X
0.500000 0.500000 0.000000  ! M
0.000000 0.000000 0.000000  ! \Gamma
0.500000 0.500000 0.500000  ! R
0.000000 0.500000 0.000000  ! X
0.500000 0.500000 0.000000  ! M
0.500000 0.500000 0.500000  ! R
&param_calc_int
/
```

3.4.5 次のステップ: RESPACK の実行

生成した入力ファイルを使った計算は次の順で実行します (コマンドのみ示します。実行方法・並列化の詳細は Quantum ESPRESSO および RESPACK のドキュメントを参照してください)。

```
$ pw.x -in scf/scf.in > scf.out                # SCF 計算
$ pw.x -in nscf/nscf.in > nscf.out             # NSCF 計算
$ qe2respack.py work/pwscf.save                # QE 出力を RESPACK 形式に変換
$ calc_wannier < input.in > log.wannier        # Wannier 関数の構成
$ calc_chiqw < input.in > log.chiqw            # cRPA 分極関数
$ calc_w3d < input.in > log.w3d                # 有効クーロン相互作用 W
$ calc_j3d < input.in > log.j3d                # 有効交換相互作用 J
```

第 4 章

コマンドリファレンス

4.1 cif2x

第一原理計算のための入力ファイルを生成する

書式:

```
cif2x [-v][-q][--dry-run] -t target input_yaml material.cif
cif2x [-v][-q][--dry-run] -t target --mp-id ID [--symprec PREC][--api-key-file␣
↪FILE] input_yaml
cif2x -h
cif2x --version
```

説明:

input_yaml に指定した入力パラメータファイルと material.cif に指定した結晶構造データを読み込み、第一原理計算プログラム用の入力ファイルを生成します。現在は Quantum ESPRESSO, VASP, OpenMX, AkaiKKR に対応しています。以下のオプションを受け付けます。

- `-v`

  実行時に表示されるメッセージを冗長にします。複数回指定すると冗長度が上がります。

- `-q`

  実行時に表示されるメッセージの冗長度を下げます。 `-v` の効果を打ち消します。複数回の指定が可能です。

- `-t` *target*

  対象となる第一原理計算プログラムを指定します。 *target* として指定可能な値は以下のとおりです。大文字小文字は区別しません。

  - `QE`, `espresso`, `quantum_espresso`: Quantum ESPRESSO 向け入力ファイルを生成します。

  - VASP: VASP 向け入力ファイルを生成します。
  - OpenMX: OpenMX 向け入力ファイルを生成します。
  - AkaiKKR: AkaiKKR 向け入力ファイルを生成します。
  - respack: RESPACK ワークフロー一式を生成します。Quantum ESPRESSO 入力 (scf/nscf タスク、および任意で bands タスク) と RESPACK 制御ファイル input.in``(``mode: respack タスク) を出力します。構造は pymatgen の高対称 k 経路が前提とする**標準プリミティブセル**である必要があります (structure.use_primitive: true、use_ibrav: false を指定し、標準プリミティブ構造を与えてください)。非立方系で不一致のセルを与えると、書き出される k 経路が QE のセルと不整合になります (cif2x は警告を出力します)。N_wannier とエネルギー窓は content/テンプレートで与える物理量で、初期ゲスは SCDM(N_initial_guess = 0、respack-wannier-py を対象) です。nscf タスクには qe2respack 用に nosym = .true./noinv = .true. を設定してください。実行順序: scf → nscf → qe2respack → respack。

- --dry-run

  生成される入力ファイルをディスクに書き込まず、標準出力に表示します。ファイルやディレクトリを作成せずに生成結果を確認したい場合に便利です。

- input_yaml

  入力パラメータファイルを指定します。形式は YAML format です。

- material.cif

  結晶構造データファイルを指定します。形式は CIF の他、pymatgen で扱える形式のファイルを指定できます。--mp-id を指定する場合は省略可能です。

- --mp-id *ID*

  material.cif を読む代わりに、Materials Project の物質 ID *ID* (例: mp-149) の結晶構造を直接取得します。material.cif と --mp-id はいずれか一方のみを指定してください。API キーは --api-key-file``(**既定** ``materials_project.key)、続いて環境変数 MP_API_KEY、pymatgen 設定ファイル (~/.config/.pmgrc.yaml) の PMG_MAPI_KEY の順に、getcif と同じ方法で解決されます。

- --symprec *PREC*

  取得した構造を書き出す際の対称性の許容誤差 (symprec、既定 0.1、Materials Project に準拠)。--symprec 0 で対称性の精密化を無効化します。--mp-id 指定時のみ有効です。

- --api-key-file *FILE*

  Materials Project の API キーを格納したファイル (# 以外の各行に 1 つ、既定 materials_project.key)。--mp-id 指定時のみ有効です。

> **注釈**
>
> `--mp-id` は、取得した構造を一時 CIF に書き出して読み直すことで「`getcif` のあと `cif2x`」の流れを再現します。Materials Project の最終構造をそのまま使用します (標準セル〔慣用セル〕への変換はしません)。CIF を経由するため、磁気モーメントなどのサイト特性は引き継がれず、無秩序 (部分占有) 構造は Quantum ESPRESSO 生成で拒否されます。これらは 2 段階の流れと同じ制限です。

- `-h`

  ヘルプを表示します。

- `--version`

  バージョン情報を表示します。

第 5 章

ファイルフォーマット

5.1 入力パラメータファイル

入力パラメータファイルでは、cif2x で第一原理計算入力ファイルを生成するための設定情報を YAML 形式で記述します。本ファイルは以下の部分から構成されます。

1. structure セクション: 結晶構造データの扱いについてのオプションを記述します。
2. optional セクション: 擬ポテンシャルファイルの指定や、YAML の参照機能を利用する場合のシンボル定義を行います。
3. tasks セクション: 入力ファイルの内容を記述します。

5.1.1 structure

`use_ibrav` (デフォルト値: `false`)

結晶構造の入力に Quantum ESPRESSO の `ibrav` パラメータを利用します。 `true` の場合、格子のとり方を Quantum ESPRESSO の convention に合うように変換します。入力ファイルにはあわせて格子に関するパラメータ a, b, c, cosab, cosac, cosbc が (必要に応じて) 書き出されます。

`tolerance` (デフォルト値: 0.01)

`use_ibrav = true` の場合に、再構成した Structure データと元データとの一致を評価する際の許容度を指定します。

`supercell` (デフォルト値: なし)

supercell を設定する場合に supercell のサイズを $[n_x, n_y, n_z]$ で指定します。例えば各軸方向に 2 倍の supercell を作成する場合は次のように指定します。

```yaml
structure:
  supercell: [2, 2, 2]
```

5.1.2 optional

第一原理計算プログラムごとに必要な global な設定を行います。記述する内容は以下の各節に記述します。

5.1.3 tasks

入力ファイルの内容を記述します。複数の入力ファイルに対応するため、`tasks` には各入力ファイルごとのブロックからなるリスト形式をとります。各ブロックに記述する項目は以下のとおりです。

`mode` (Quantum ESPRESSO)

入力ファイルの計算内容を指定します。現時点では Quantum ESPRESSO の pw.x 向けに `scf` と `nscf` に対応しています。対応していない mode については、 `content` の内容をそのまま出力します。

`output_file` (Quantum ESPRESSO)

出力ファイル名を指定します。

`output_dir`

出力先のディレクトリ名を指定します。デフォルト値はカレントディレクトリです。

`content`

出力内容を指定します。Quantum ESPRESSO の場合は `namelist` ブロックに namelist データ (`&system` や `&control` など) を記述し、 `K_POINTS` 等の card データを個別のブロックとして記述します。card データはパラメータをとるものがあります。

`template` (Quantum ESPRESSO)

`template_dir` (VASP)

出力内容のテンプレートファイルを指定します。指定がない場合はテンプレートを利用しません。このファイルの内容を `content` に追加します。同じデータがある場合は後者を優先します。

5.1.4 パラメータセット指定

入力パラメータには値のリストや範囲を指定することができ、値の組み合わせごとに個別のディレクトリを作成して入力ファイルを生成します。パラメータセットの指定には特別な構文 `${...}` を用います。指定方法は次の通りです:

- リスト `${[ A, B, ... ]}`

  パラメータセットを Python のリストとして列挙します。各項目はスカラー値やリストを指定できます。

- 範囲指定 `${range(N)}`, `${range(start, end, step)}`

  パラメータの範囲を指定します。それぞれ 0〜N-1, start〜end を step 刻み (step を省略した場合は 1) です。`N`, `start`, `end`, `step` は int または float です。

5.2 Quantum ESPRESSO 向けパラメータ

optional セクションおよび tasks セクションの content に記載する内容について、Quantum ESPRESSO 固有の内容を記述します。現時点では pw.x の scf および nscf に対応しています。

5.2.1 optional セクション

pp_file

元素種と擬ポテンシャルを対応付ける CSV 形式のインデックスファイルを指定します。ファイルの 1 行目はヘッダで、列は element,pseudopotential,nexclude,orbitals です。各列の意味は以下のとおりです。

- element: 元素記号。
- pseudopotential: 擬ポテンシャルファイル名のうち、元素記号と拡張子 .UPF を除いた部分 (name stem)。擬ポテンシャルファイル名は {element}.{pseudopotential}.UPF として組み立てられます (スピン軌道相互作用を含む非共線計算では、生成される ATOMIC_SPECIES のファイル名が {element}.rel-{pseudopotential}.UPF のように rel- 付きになります)。
- nexclude: Wannier 関数によるバンド数の見積り (nbnd) で除外する内殻バンド数。
- orbitals: Wannier 化の対象とする軌道を s, p, d, f の文字で並べたもの。nbnd の計算に用いられます (src/cif2x/struct2qe.py の _find_nbnd_info)。

例:

```
element,pseudopotential,nexclude,orbitals
Fe,pbe-spn-rrkjus_psl.0.2.1,4,spd
```

この行は擬ポテンシャルファイル Fe.pbe-spn-rrkjus_psl.0.2.1.UPF に対応します。

cutoff_file

擬ポテンシャルファイルとカットオフを対応付ける CSV 形式のインデックスファイルを指定します。列は pseudofile,ecutwfc,ecutrho で、それぞれ擬ポテンシャルファイル名、 ecutwfc の値、ecutrho の値です。

pseudo_dir

擬ポテンシャルファイルを格納するディレクトリ名を指定します。カットオフの値を擬ポテンシャルファイルから取得する場合に使用します。Quantum ESPRESSO の pseudo_dir パラメータとは独立に指定します。

5.2.2 content

namelist

- structure セクションの use_ibrav パラメータに応じて、 &system の格子情報の指定が上書きされます。
  - use_ibrav = false: ibrav は 0 にセットされます。また、格子パラメータに関する a, b, c, cosab, cosac, cosbc, celldm は削除されます。
  - use_ibrav = true: ibrav は結晶構造データから取得された Bravais 格子のインデックスがセットされます。また、Structure データは基本格子のとり方など Quantum ESPRESSO の convention に合わせて再構成されます。
- nat (原子数) および ntyp (元素種の数) は結晶構造データから取得される値で上書きされます。
- カットオフ ecutwfc および ecutrho の情報は、パラメータの値が空欄の場合は擬ポテンシャルファイルから取得します。

CELL_PARAMETERS

- structure セクションの use_ibrav パラメータが true の場合は出力されません。false の場合は格子ベクトルが出力されます。単位は angstrom です。
- 格子ベクトルの情報は結晶構造データから取得されます。 data フィールドに 3x3 の行列を直接指定した場合はその値が用いられます。

ATOMIC_SPECIES

- 原子種・原子量・擬ポテンシャルファイル名のリストを出力します。
- 原子種の情報は結晶構造データから取得されます。擬ポテンシャルのファイル名は pp_file で指定する CSV 形式のインデックスファイルを参照します。
- data フィールドに必要なデータを指定した場合はその値が用いられます。

ATOMIC_POSITIONS

- 原子種と原子座標 (fractional coordinate) のリストを出力します。
- ignore_species に原子種または原子種のリストを指定した場合、その原子種については if_pos の値が 0 にセットされます。MD や構造最適化の際に使われます。
- data フィールドに必要なデータを指定した場合はその値が用いられます。

K_POINTS

- k 点の情報を出力します。 option に出力タイプを指定します。
  - gamma: Γ 点を用います。

  - crystal: メッシュ状の k 点のリストを出力します。メッシュの指定は grid パラメータまたは vol_density や k_resolution から導出される値が用いられます。
  - automatic: k 点のメッシュを指定します。メッシュの指定は grid パラメータまたは vol_density や k_resolution から導出される値が用いられます。 シフトの指定は kshifts パラメータを参照します。

- メッシュの指定は以下の順序で決定されます。
  - grid パラメータの指定。grid の値は n_x, n_y, n_z の配列またはスカラー値 n です。後者の場合は $n_x = n_y = n_z = n$ と仮定します。
  - vol_density パラメータから自動導出。
  - k_resolution パラメータから自動導出。k_resolution のデフォルトは 0.15 です。
- data フィールドに必要なデータを指定した場合はその値が用いられます。

5.3 VASP 向けパラメータ

optional セクションおよび tasks セクションの content に記載する内容について、VASP 固有の内容を記述します。

5.3.1 optional

擬ポテンシャルのタイプや格納場所を指定します。

pymatgen では、擬ポテンシャルファイルを PMG_VASP_PSP_DIR/*functional*/POTCAR. *element* (.gz) または PMG_VASP_PSP_DIR/*functional*/ *element* /POTCAR から取得します。PMG_VASP_PSP_DIR はディレクトリの指定で、設定ファイル ~/.config/.pmgrc.yaml に記載するか、同名の環境変数に指定します。また、*functional* は擬ポテンシャルのタイプで、 POT_GGA_PAW_PBE や POT_LDA_PAW などが決められています。

pseudo_functional

擬ポテンシャルのタイプを指定します。タイプの指定と上記の *functional* の対応は pymatgen 内のテーブルに定義され、 PBE → POT_GGA_PAW_PBE, LDA → POT_LDA_PAW などのようになっています。

以下の pseudo_dir を指定した場合は pymatgen の流儀を無視して擬ポテンシャルの格納ディレクトリを探します。

pseudo_dir

擬ポテンシャルの格納ディレクトリを指定します。擬ポテンシャルファイルのファイル名は pseudo_dir/POTCAR. *element* (.gz) または pseudo_dir/*element*/POTCAR です。

5.3.2 tasks

テンプレートファイルは、 `template_dir` で指定するディレクトリ内に `INCAR`, `KPOINTS`, `POSCAR`, `POTCAR` ファイルを配置します。ファイルがない項目は無視されます。

5.3.3 content

incar

- INCAR ファイルに記述するパラメータを列挙します。

kpoints

- `type`

  KPOINTS の指定方法を記述します。以下の値に対応しています。タイプによりパラメータが指定可能なものがあります。詳細は pymatgen.io.vasp のマニュアルを参照してください。

  - `automatic`

    parameter: `grid`

  - `gamma_automatic`

    parameter: `grid`, `shift`

  - `monkhorst_automatic`

    parameter: `grid`, `shift`

  - `automatic_density`

    parameter: `kppa`, `force_gamma`

  - `automatic_gamma_density`

    parameter: `grid_density`

  - `automatic_density_by_vol`

    parameter: `grid_density`, `force_gamma`

  - `automatic_density_by_lengths`

    parameter: `length_density`, `force_gamma`

  - `automatic_linemode`

    parameter: `division`, `path_type` (HighSymmKpath の path_type に対応)

5.4 OpenMX 向けパラメータ

optional セクションおよび tasks セクションの content に記載する内容について、OpenMX 固有の内容を記述します。

5.4.1 optional

data_path

擬原子軌道および擬ポテンシャルのファイルを格納するディレクトリを指定します。入力ファイルの DATA.PATH パラメータに対応します。

5.4.2 content

precision

擬原子軌道を OpenMX マニュアル 10.6 章の Table 1, 2 にしたがって選択します。 quick, standard, precise のいずれかの値を取ります。デフォルト値は quick です。

5.5 AkaiKKR 向けパラメータ

optional セクションおよび tasks セクションの content に記載する内容について、AkaiKKR 固有の内容を記述します。

5.5.1 optional

workdir

一時ファイルの出力先を指定します。指定しない場合は /tmp または TMPDIR 環境変数の値を利用します。

5.5.2 content

content には AkaiKKR の入力パラメータの内容を記述します。指定のない項目は空欄が出力され、AkaiKKR 内部のデフォルト値が使われます。以下のパラメータは結晶構造データから決まる値で置き換えられます。

- brvtyp: ただし、 brvtyp に aux (を含む) 値が指定されている場合は上書きされません。
- 格子パラメータ: a, c/a, b/a, alpha, beta, gamma, r1, r2, r3
- タイプ情報: ntyp, type, ncmp, rmt, field, mxl, anclr, conc
- 元素種の情報: natm, atmicx, atmtyp

なお、 rmt と field の値は、入力パラメータファイル内で ntyp 個の要素からなるリストとして指定されている場合のみ、入力パラメータファイルの値が使われます。

第 6 章

拡張ガイド

6.1 Quantum ESPRESSO の mode を追加する

Quantum ESPRESSO の計算モードへの対応を追加するには、 `src/cif2x/qe/calc_mode.py` の `create_modeproc()` 関数に mode と変換クラスの対応付けを記述します。

```
def create_modeproc(mode, qe):
    if mode in ["scf", "nscf", "relax", "vc-relax", "bands"]:
        modeproc = QEmode_pw(qe)
    else:
        modeproc = QEmode_generic(qe)
    return modeproc
```

mode ごとの変換機能は `QEmode_base` の派生クラスとしてまとめられています。このクラスは `update_namelist()` で namelist ブロックの更新と、`update_cards()` で cards ブロックのデータ生成を行います。現在は pw.x の scf、nscf、relax、vc-relax、bands に対応する `QEmode_pw` クラスと、変換せずそのまま出力する `QEmode_generic` クラスが用意されています。

```
class QEmode_base:
    def __init__(self, qe):
    def update_namelist(self, content):
    def update_cards(self, content):
```

namelist については、空欄の値を結晶構造データ等から生成して代入するほか、格子パラメータなど Structure から決まる値や、他のパラメータとの整合性をとる必要のある値を強制的にセットする場合があります。処理内容はモードごとに個別に対応します。

cards ブロックについては、card の種類ごとに関数を用意し、card 名と関数の対応付けを `card_table` 変数に列挙します。基底クラスの `update_cards()` では、card 名から対応する関数を取得して実行し、card の情報を更新します。もちろん、全く独自に `update_cards()` 関数を作成することもできます。

```
self.card_table = {
    'CELL_PARAMETERS': generate_cell_parameters,
    'ATOMIC_SPECIES': generate_atomic_species,
    'ATOMIC_POSITIONS': generate_atomic_positions,
    'K_POINTS': generate_k_points,
}
```

card ごとの関数は src/cif2x/qe/cards.py にまとめられており、関数名は generate_{card 名} としています。この関数は card ブロックのパラメータを引数に取り、card 名、option、data フィールドからなる辞書データを返します。

6.2 対応する計算モード

QEmode_pw は scf、nscf、relax、vc-relax、bands について構造カード (CELL_PARAMETERS、ATOMIC_SPECIES、ATOMIC_POSITIONS、K_POINTS) を生成し、calculation をタスクの mode から設定します。&ions``(``relax/vc-relax)・&cell``(``vc-relax) ネームリストはテンプレート/content の記述をそのまま使用します。これらが無いと pw.x はエラーになるため、必ず記述してください。

bands では content の K_POINTS オプションに crystal_b を指定します。経路は結晶対称性から pymatgen の高対称 k 経路を用いて生成されます。例:

```
tasks:
  - mode: bands
    output_file: bands.in
    content:
      system:
        nbnd: 16         # 伝導帯が必要な場合は明示的に指定
      K_POINTS:
        option: crystal_b
        line_npoints: 30   # 高対称点間に生成する点数
```

line_npoints の既定値は 20 です。高対称経路に不連続 (分断された区間) が含まれる場合、各区間終端の整数を 0 にすることで bands.x が区間をまたいで補間しないようにします。独自経路を使う場合は path にラベル列のリストを与えます (ラベルは自動生成された k 経路に含まれる必要があります)。

Quantum ESPRESSO は既定で占有 (価電子) バンドのみを計算するため、伝導帯をバンド構造に含めるには content.system で nbnd を明示的に指定してください。

高対称点の座標は**標準プリミティブセル**で定義されるため、bands では構造がそのセルである必要があります。use_primitive: true``(かつ ``use_ibrav: false) を指定してください。慣用セルやスーパーセルなど非標準の入力の場合、cif2x は警告 (「band path: ... the path may be incorrect」) を出力し、サンプリングされる経路は出力される CELL_PARAMETERS と一致しません。

pw.x に calculation='dos' はありません。状態密度計算は scf -> nscf``(**密なメッシュ**)-> ``dos``(``dos.x 入力) の連鎖です。dos タスクは QEmode_generic でそのまま出力されるため、&DOS ネームリストはテンプレート/content に記述します:

```
tasks:
  - mode: scf
    output_file: scf.in
    content: { K_POINTS: { option: automatic, grid: [8, 8, 8] } }
  - mode: nscf
    output_file: nscf.in
    content: { K_POINTS: { option: automatic, grid: [16, 16, 16] } }
  - mode: dos
    template: dos.in_tmpl     # &DOS ネームリストを含む
    output_file: dos.in
```

生成した入力は scf → nscf → dos.x の順に実行し、各ステップが前のステップの保存データを読めるよう、3 つすべてで同じ prefix と outdir を指定してください。

6.3 トラブルシューティング

cif2x 実行時に起こりやすいエラーと対処法を以下にまとめます。

cif2x は入力ファイルを生成する前に、ターゲット名と input.yaml を検証します。ターゲット名や入力が不正な場合は、エラーメッセージを 1 行出力して終了コード 1 で終了します (Python のトレースバックは表示されません)。

- **unsupported target '...'**

  -t/--target の値が quantum_espresso (qe, espresso)、 vasp、 openmx、 akaikkr のいずれでもない場合に出力されます。メッセージに有効な選択肢が列挙されます。

- **task N: 'mode' is required for target 'quantum_espresso'**

  Quantum ESPRESSO 向けの実行で、 tasks の各要素に mode が指定されていない場合に出力されます。各 task に mode (scf, nscf など) を指定してください。

- **task N: 'output_file' is required for target '...'**

  出力ファイル名が必要な task に output_file が指定されていない場合に出力されます。各 task に出力ファイル名を output_file で指定してください (quantum_espresso、 openmx、 akaikkr で必須です)。

- **task N: unknown key '...' for target '...'**

  task に未知のキー (例: output_fil のようなタイプミス) が含まれる場合に出力されます。メッセージにそのターゲットで許可されるキーが列挙されます。なお、自由記述ブロック (content、 optional、 structure) のキーは検証対象外です。

- **pp_file / cutoff_file / pseudo_dir not specified または not found の警告**

  optional セクションでこれらのパラメータが指定されていない、または指定したパスが存在しない場合に警告が出力されます (処理は継続します)。擬ポテンシャルのインデックスファイル (pp_file)、カットオフのインデックスファイル (cutoff_file)、擬ポテンシャル格納ディレクトリ (pseudo_dir) のパスが正しいか確認してください。なお、 cutoff_file / pseudo_dir は警告のみで処理が継続しますが、 pp_file は ATOMIC_SPECIES の生成や nbnd の自動設定で実際に参照されるため事実上必須であり、未指定のままだと処理の後段で RuntimeError となります。

- **カットオフが 0.0 になる**

  ecutwfc / ecutrho を空欄にして自動取得させる場合、(その元素について pp_file の対応付けが存在する前提で) 対応する .UPF ファイルが pseudo_dir に見つからず、かつ cutoff_file にも該当エントリがないと、カットオフの値が 0.0 にフォールバックします。擬ポテンシャルファイルやカットオフのインデックスエントリが揃っているか確認してください。(なお、 pp_file にその元素の行自体が無い場合は 0.0 フォールバックではなく KeyError で停止します。)

第 7 章

CIF データ取得ツール (getcif)

7.1 概要

getcif は物質材料データベースから結晶構造データ等を取得するツールです。現在は Materials Project からのデータ取得に対応しています。物質の組成や対称性、バンドギャップなどの物性値をもとにデータベースを検索し、データを取得することができます。

7.2 チュートリアル

結晶構造などのデータを物質材料データベースから取得するツール getcif を使うには、検索条件と取得するデータを記述した入力パラメータファイルを作成し、プログラム getcif を実行します。現在は Materials Project が公開しているデータベースに対応しています。以下では `docs/tutorial/getcif` ディレクトリにある ABO3 系の物質を検索・取得するサンプルを例にチュートリアルを実施します。

7.2.1 API キーを取得する

Materials Project のデータベースをプログラムから検索するには、あらかじめ Materials Project にユーザ登録し、API キーを取得する必要があります。Materials Project の公式サイト https://next-gen.materialsproject.org にアクセスし、Login します。API キーはユーザ登録時に自動的生成され、ユーザのダッシュボードから確認できます。取得した API キーは安全に保管し、他人に知られないようにしましょう。

API キーを getcif にセットするには、以下のいずれかを実行します。

(a) pymatgen の設定ファイルに登録する

```
$ pmg config --add PMG_MAPI_KEY <API_KEY>
```

を実行するか、設定ファイル `~/.config/.pmgrc.yaml` に

```
PMG_MAPI_KEY: <API_KEY>
```

を書き込みます。

(b) 環境変数にセットする

```
$ MP_API_KEY="<API_KEY>"
$ export MP_API_KEY
```

を実行します。

(c) ファイルに格納する

API キーをファイルに書き込み、getcif を実行するディレクトリに配置します。ファイル名のデフォルトは materials_project.key です。異なるファイル名を使う場合は、入力パラメータファイル (input.yaml) の api_key_file にファイル名を指定します。ファイル名は .key の拡張子が必要です。

```
database:
  api_key_file: materials_project.key
```

註: バージョン管理ツールを使っている場合は、 .key の拡張子を持つファイルを管理から除外するとよいでしょう。(Git の場合は .gitignore ファイルに *.key を追加します。)

7.2.2 入力パラメータファイルを作成する

入力パラメータファイルにはデータベース検索および出力についての設定を記述します。

以下に入力パラメータファイルのサンプルを記載します。このファイルは YAML 形式のテキストファイルで、データベースへの接続に必要な情報や、検索条件、取得するデータの種類などの内容を記述します。仕様の詳細については **ファイルフォーマット** の章を参照してください。

YAML フォーマットでは、 keyword: value の辞書形式でパラメータを記述します。 value には数値や文字列などのスカラー値や、複数の値を [ ] または箇条書きの形式で列挙するリスト型、または辞書型を入れ子にすることができます。検索条件と出力項目については、特別な記法として、リスト型を括弧を使わず空白区切りで列挙する形式でも書くことができます。

```
database:
  target: materials project

option:
  output_dir: result
  # dry_run: false

properties:
  band_gap: < 1.0
  is_stable: true
  is_metal: false
```

(次のページに続く)

(前のページからの続き)

```
  formula: "**O3"
  spacegroup_symbol: Pm-3m

fields: |
  structure
  band_gap
  symmetry
```

入力パラメータファイルは database, option, properties, fields のブロックからなります。database にはデータベース接続に関する設定を記述します。例では target に Materials Project を指定していますが、現時点ではこの項目は無視されます。API キーをファイルから読み込ませる場合は、 api_key_file にファイル名 (.key 拡張子) を指定します。デフォルトのファイル名は materials_project.key です。なお、 database セクションに API キーそのものを直接指定する項目はありません。API キーの与え方には、この api_key_file のほかに、環境変数 MP_API_KEY を利用する方法や、pymatgen の設定ファイル (~/.config/.pmgrc.yaml) に PMG_MAPI_KEY として登録する方法があります (前述の「API キーを取得する」を参照)。例では api_key_file を指定せず、環境変数または pymatgen の設定ファイルを用いる方式を仮定しています。

option には実行時のオプションを記述します。 output_dir は取得したデータの格納先ディレクトリを指定します。省略時にはカレントディレクトリに書き出されます。 dry_run を true にセットすると、データベースへの接続はせず、検索条件を出力して終了します。 dry_run はコマンドラインオプションでも指定できます。

properties は検索条件の指定を行います。検索条件を「項目: 値」の書式で列挙し、これらの条件は AND で扱われます。例では、バンドギャップが 1.0 以下、安定な絶縁体で、組成式は ABO3 (A, B は任意の元素種)、空間群は Pm-3m という条件を指定しています。 band_gap には値の範囲を上限・下限の組で指定するほか、 < 1.0 のような記法も使用できます。検索条件にどのような項目が指定できるかは Appendix を参照してください。

fields には出力項目を列挙します。YAML のリスト形式のほか、空白区切りで項目を列挙する書き方もできます。また、例に示したとおり、YAML の記法を使って複数行で書くこともできます。structure は結晶構造データで、取得したデータは CIF 形式で出力されます。 band_gap はバンドギャップの数値、 symmetry は対称性の情報です。この他に、 material_id で Materials Project 内の物質データのインデックスと、 formula_pretty で組成式が暗黙的に取得されます。出力項目の一覧は Appendix を参照してください。また、getcif コマンドのヘルプメッセージにも一覧が出力されます。

7.2.3 データを取得する

入力パラメータファイル (input.yaml) を引数として getcif を実行します。

```
$ getcif input.yaml
```

getcif を実行すると Materials Project のデータベースに接続し、検索条件に合致するデータを取得します。標準出力には、以下のように、物質の material ID と組成式、データ項目のサマリーが出力されます。

```
material_id  formula  band_gap  symmetry  formula_pretty
mp-861502  AcFeO3  0.9887999999999995  crystal_system=<CrystalSystem.cubic: 'Cubic'>␣
↪symbol='Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  AcFeO3
mp-977455  PaAgO3  0.915  crystal_system=<CrystalSystem.cubic: 'Cubic'> symbol='Pm-3m'␣
↪number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  PaAgO3
mp-11775  RbUO3  0.45420000000000016  crystal_system=<CrystalSystem.cubic: 'Cubic'>␣
↪symbol='Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  RbUO3
mp-3163  BaSnO3  0.37239999999999984  crystal_system=<CrystalSystem.cubic: 'Cubic'>␣
↪symbol='Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  BaSnO3
mp-4126  KUO3  0.44540000000000024  crystal_system=<CrystalSystem.cubic: 'Cubic'> symbol=
↪'Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  KUO3
mp-865322  UTlO3  0.27360000000000007  crystal_system=<CrystalSystem.cubic: 'Cubic'>␣
↪symbol='Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  UTlO3
mp-753781  EuHfO3  0.47959999999999996  crystal_system=<CrystalSystem.cubic: 'Cubic'>␣
↪symbol='Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  EuHfO3
```

取得したデータは、output_dir で指定した result ディレクトリ内に物質ごとに格納されます。この例では、result 以下に mp-3163 から mp-977455 までの 7 つのディレクトリが作成され、各ディレクトリには次のファイルが書き込まれます。

- band_gap

  バンドギャップの値

- formula

  組成式 (formula_pretty に対応します)

- structure.cif

  CIF 形式の結晶構造データ

- symmetry

  対称性に関するデータ

getcif の実行に --dry-run オプションを付けると、以下のように検索条件を出力して終了します。データベースに実際に接続する前に検索項目を確認できます。

```
$ getcif --dry-run input.yaml
{'band_gap': (None, 1.0), 'is_stable': True, 'is_metal': False, 'formula': '**O3',
 'spacegroup_symbol': 'Pm-3m', 'fields': ['structure', 'band_gap', 'symmetry', 'material_
↪id', 'formula_pretty']}
```

7.3 コマンドリファレンス

7.3.1 getcif

結晶構造データをデータベースから取得する

書式:

```
getcif [-v][-q] [--dry-run] input_yaml
getcif -h
getcif --version
```

説明:

input_yaml に指定した入力パラメータファイルを読み込み、データベースを検索して結晶構造等のデータを取得します。以下のオプションを受け付けます。

- `-v`

  実行時に表示されるメッセージを冗長にします。複数回指定すると冗長度が上がります。

- `-q`

  実行時に表示されるメッセージの冗長度を下げます。 `-v` の効果を打ち消します。複数回の指定が可能です。

- `--dry-run`

  検索パラメータを出力し、データベースへの接続をせずに終了します。検索条件を確認することができます。入力パラメータファイルの `dry_run` の指定より優先します。

- `input_yaml`

  入力パラメータファイルを指定します。ファイルフォーマットは YAML 形式です。

- `-h`, `--help`

  ヘルプを表示します。指定可能な検索条件 (properties) と取得データ (fields) の一覧も表示されます。

- `--version`

  バージョン情報を表示します。

7.4 ファイルフォーマット

7.4.1 入力パラメータファイル

入力パラメータファイルでは、getcif で Materials Project の物質材料データベースから結晶構造等のデータを取得するための設定情報を YAML 形式で記述します。本ファイルは以下の部分から構成されます。

1. database セクション: 接続するデータベースについての情報を記述します。
2. option セクション: 出力先のディレクトリや実行条件などを記述します。
3. properties セクション: 検索条件を記述します。
4. fields セクション: 取得データの種類を記述します。

database

`target`

接続先のデータベースを指定します。現在はこの項目は無視されます。

`api_key_file` (デフォルト値: `materials_project.key`)

データベースに接続する際の API キーを格納したファイルのファイル名を指定します。ファイル名の拡張子は `.key` とします。`.key` ファイルが存在する場合はそれが優先され、先頭のコメント行以外の行が使われます (空行はスキップされないため、先頭の非コメント行は空であってはいけません)。ファイルが存在しない、または先頭の非コメント行が空の場合は、環境変数 `MP_API_KEY` または pymatgen の設定ファイル `~/.config/.pmgrc.yaml` の `PMG_MAPI_KEY` から API キーを取得します。

API キーファイルはテキスト形式です。`#` から始まる行はコメントとして扱われます。前後の空白は無視されます。複数行からなる場合は最初の有効な行からキーを取得します。

option

`output_dir` (デフォルト値: `""`)

取得データを格納するディレクトリを指定します。データは `output_dir` 以下に、material ID をディレクトリ名としたディレクトリに出力されます。指定がない場合はカレントディレクトリです。

`dry_run` (デフォルト値: `False`)

データベースへの接続は行わず、検索条件を出力して終了します。検索内容の確認を行うことができます。

`symprec` (デフォルト値: 0.1)

結晶構造データを CIF ファイルに出力する際の対称性を判定する許容精度を指定します。デフォルトは 0.1 です。 `symprec` に 0.0 を指定した場合は `symprec` を指定しないものとして扱い、対称性を考慮しない CIF ファイルが生成されます。

`symprec` は、結晶構造における対称性を判定する際の許容精度 (tolerance) を指定するパラメータです。対称性の計算においては、原子位置の微細なずれや数値計算の精度の影響を考慮する必要があります。`symprec` はこのずれの許容範囲を制御し、対称操作が適用されるかどうかを決定する際の閾値として機能します。

`symprec` を小さく設定する (例: 0.01) と、対称性の判定がより厳密になり、結晶構造のわずかなずれでも対称操作が適用されない可能性が高まります。その結果、より低い対称性の空間群が得られることが

あります。逆に、symprec を大きく設定する (例: 1.0) と、対称性の判定が緩やかになり、わずかなずれが無視され、より高い対称性が認められることがあります。

なお、 fields セクションで symmetry を指定すると、Materials Project でデフォルトの symprec=0.1 で判定された対称性の情報を取得し、テキストファイル (symmetry) に出力します。

properties

検索条件を記述します。元素組成や結晶の対称性、物性値の範囲などの項目を、「項目名: 値」の形式で指定します。これらの条件は AND で扱われます。指定できる項目は Materials Project の API に定義されていますが、指定方法は mp-api ライブラリの materials.summary.search のパラメータに準拠します。項目のリストは Appendix を参照してください。また、 getcif --help で一覧を見ることができます。

値の指定方法は次のとおりです。YAML 形式に準拠しますが、一部に簡便な記法を用意しています。

- 数値、文字列

  そのまま記述します。

- 真偽値

  true または false を記述します

- 数値や文字列のリスト

  YAML 形式の箇条書きおよび [...] にカンマ区切りで記述するほか、空白区切りで列挙する記法も可能です。例:

  ```
  element: Sr Ti
  ```

- 数値の範囲

  上限・下限のリストとして [min, max] のように記述するほか、空白区切りで min max のように記述することもできます。また、以下の記法も可能です。

  <= max
  : max 以下

  < max
  : max より小さい (実数の場合は <= と同等。整数の場合は <= max-1 として扱われる)

  >= min
  : min 以上

  > min
  : min より大きい (実数の場合は >= と同等。整数の場合は >= min+1 として扱われる)

  min ~ max
  : min 以上 max 以下

注記:

- 記号と数値の間は空白を置きます。
- YAML 記法では > は特殊文字として扱われるため、 >= min, > min はそれぞれ ">= min", "> min" のように " " で囲む必要があります。
- リストで記述する場合、 <= max, >= min はそれぞれ [None, max], [min, None] のように表記します。

- ワイルドカード

formula **には元素種にワイルドカード * を指定できます。その場合は値を " " で囲みます。例**:

```
formula: "**O3"
```

ABO_3 系の物質を指定します。

fields

取得するデータの種類を記述します。項目のリストを YAML 形式で列挙するほか、空白区切りの文字列として記述することもできます。文字列は YAML 記法 | を用いて複数行で書くこともできます。指定できる項目は Materials Project の API の fields パラメータに準拠します。項目のリストは Appendix を参照してください。また、 getcif --help で一覧を見ることができます。

material_id と formula_pretty は暗黙的に取得します。

取得したデータは、 option セクションの output_dir で指定したディレクトリ内に、物質ごとに material_id をディレクトリ名とするディレクトリを作成し、その中に格納されます。項目ごとに、項目名をファイル名としたファイルに保存されます。但し、結晶構造データ (structure) は structure.cif というファイル名で CIF 形式で書き出されます。

7.5 パラメータリスト

7.5.1 検索条件 (properties)

properties に指定できる項目と、その項目がどのような値を取るかを以下にまとめます。

Materials Project API のクライアントアプリケーションの一つとして mp-api パッケージが Materials Project から公開されており、getcif はこのライブラリを利用してデータベースへの接続を行います。以下は MPRester クラスの materials.summary.search メソッドのパラメータに対応します。(以下の表は materials.summary.search のソースコードのコメントから転記しました。)

値の型の表記は次のとおりです。

- str: 文字列型
- List[str]: 文字列型のリスト

- `str | List[str]`: 単一の文字列、または、文字列型のリスト
- `int`: 整数型
- `bool`: 真偽値 (true または false)
- `Tuple[float,float]`: 実数値 2 つからなる組 (リスト)
- `Tuple[int,int]`: 整数値 2 つからなる組 (リスト)
- `CrystalSystem`: 結晶のタイプを表す文字列。Triclinic, Monoclinic, Orthorhombic, Tetragonal, Trigonal, Hexagonal, Cubic のいずれか。
- `List[HasProps]`: 特性値のタイプを表す文字列のリスト。特性値は emmet.core.summary に定義されている。以下のいずれかの値を取る。

  absorption, bandstructure, charge_density, chemenv, dielectric, dos, elasticity, electronic_structure, eos, grain_boundaries, insertion_electrodes, magnetism, materials, oxi_states, phonon, piezoelectric, provenance, substrates, surface_properties, thermo, xas

- `Ordering`: 磁気秩序を表す文字列。FM, AFM, FiM, NM のいずれか。

値のリストは、YAML 形式の箇条書きおよび `[ ... ]` にカンマ区切りで記述するほか、空白区切りで列挙する記法も可能です。

`Tuple` で表される型は値の範囲 (min, max) の指定に使われます。値のリストとして記述するほか、空白区切りで `min max` のように記述することもできます。また、以下の表記も可能です。

`<= max` : max 以下

`< max` : max より小さい

`>= min` : min 以上

`> min` : min より大きい

`min ~ max` : min 以上 max 以下

表 7.1: 検索条件のキーワード

Keyword	Type	Description
band_gap	Tuple[float,float]	Minimum and maximum band gap in eV to consider.
chemsys	str | List[str]	A chemical system, list of chemical systems (e.g., Li-Fe-O, Si-*, [Si-O, Li-Fe-P]), or single formula (e.g., Fe2O3, Si*).
crystal_system	CrystalSystem	Crystal system of material.
density	Tuple[float,float]	Minimum and maximum density to consider.
deprecated	bool	Whether the material is tagged as deprecated.

次のページに続く

表 7.1 – 前のページからの続き

Keyword	Type	Description
e_electronic	Tuple[float,float]	Minimum and maximum electronic dielectric constant to consider.
e_ionic	Tuple[float,float]	Minimum and maximum ionic dielectric constant to consider.
e_total	Tuple[float,float]	Minimum and maximum total dielectric constant to consider.
efermi	Tuple[float,float]	Minimum and maximum fermi energy in eV to consider.
elastic_anisotropy	Tuple[float,float]	Minimum and maximum value to consider for the elastic anisotropy.
elements	List[str]	A list of elements.
energy_above_hull	Tuple[float,float]	Minimum and maximum energy above the hull in eV/atom to consider.
equilibrium_reaction_energy	Tuple[float,float]	Minimum and maximum equilibrium reaction energy in eV/atom to consider.
exclude_elements	List[str]	List of elements to exclude.
formation_energy	Tuple[float,float]	Minimum and maximum formation energy in eV/atom to consider.
formula	str | List[str]	A formula including anonymized formula or wild cards (e.g., Fe2O3, ABO3, Si*). A list of chemical formulas can also be passed (e.g., [Fe2O3, ABO3]).
g_reuss	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Reuss average of the shear modulus.
g_voigt	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Voigt average of the shear modulus.
g_vrh	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Voigt-Reuss-Hill average of the shear modulus.
has_props	List[HasProps]	The calculated properties available for the material.
has_reconstructed	bool	Whether the entry has any reconstructed surfaces.
is_gap_direct	bool	Whether the material has a direct band gap.
is_metal	bool	Whether the material is considered a metal.
is_stable	bool	Whether the material lies on the convex energy hull.
k_reuss	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Reuss average of the bulk modulus.
k_voigt	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Voigt average of the bulk modulus.
k_vrh	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Voigt-Reuss-Hill average of the bulk modulus.
magnetic_ordering	Ordering	Magnetic ordering of the material.
material_ids	List[str]	List of Materials Project IDs to return data for.

次のページに続く

表 7.1 – 前のページからの続き

Keyword	Type	Description
n	Tuple[float,float]	Minimum and maximum refractive index to consider.
num_elements	Tuple[int,int]	Minimum and maximum number of elements to consider.
num_sites	Tuple[int,int]	Minimum and maximum number of sites to consider.
num_magnetic_sites	Tuple[int,int]	Minimum and maximum number of magnetic sites to consider.
num_unique_magnetic_sites	Tuple[int,int]	Minimum and maximum number of unique magnetic sites to consider.
piezoelectric_modulus	Tuple[float,float]	Minimum and maximum piezoelectric modulus to consider.
poisson_ratio	Tuple[float,float]	Minimum and maximum value to consider for Poisson's ratio.
possible_species	List[str]	List of element symbols appended with oxidation states. (e.g. Cr2+,O2-)
shape_factor	Tuple[float,float]	Minimum and maximum shape factor values to consider.
spacegroup_number	int	Space group number of material.
spacegroup_symbol	str	Space group symbol of the material in international short symbol notation.
surface_energy_anisotropy	Tuple[float,float]	Minimum and maximum surface energy anisotropy values to consider.
theoretical	bool	Whether the material is theoretical.
total_energy	Tuple[float,float]	Minimum and maximum corrected total energy in eV/atom to consider.
total_magnetization	Tuple[float,float]	Minimum and maximum total magnetization values to consider.
total_magnetization_normalized	Tuple[float,float]	Minimum and maximum total magnetization values normalized by formula units to consider.
total_magnetization_normalized	Tuple[float,float]	Minimum and maximum total magnetization values normalized by volume to consider.
uncorrected_energy	Tuple[float,float]	Minimum and maximum uncorrected total energy in eV/atom to consider.
volume	Tuple[float,float]	Minimum and maximum volume to consider.
weighted_surface_energy	Tuple[float,float]	Minimum and maximum weighted surface energy in J/m^2 to consider.
weighted_work_function	Tuple[float,float]	Minimum and maximum weighted work function in eV to consider.

7.5.2 出力項目 (fields)

fields に指定できる項目を以下に列挙します。

```
band_gap
bandstructure
builder_meta
bulk_modulus
cbm
chemsys
composition
composition_reduced
database_IDs
decomposes_to
density
density_atomic
deprecated
deprecation_reasons
dos
dos_energy_down
dos_energy_up
e_electronic
e_ij_max
e_ionic
e_total
efermi
elements
energy_above_hull
energy_per_atom
equilibrium_reaction_energy_per_atom
es_source_calc_id
formation_energy_per_atom
formula_anonymous
formula_pretty
grain_boundaries
has_props
has_reconstructed
homogeneous_poisson
is_gap_direct
is_magnetic
is_metal
```

(次のページに続く)

(前のページからの続き)

```
is_stable
last_updated
material_id
n
nelements
nsites
num_magnetic_sites
num_unique_magnetic_sites
ordering
origins
possible_species
property_name
shape_factor
shear_modulus
structure
surface_anisotropy
symmetry
task_ids
theoretical
total_magnetization
total_magnetization_normalized_formula_units
total_magnetization_normalized_vol
types_of_magnetic_species
uncorrected_energy_per_atom
universal_anisotropy
vbm
volume
warnings
weighted_surface_energy
weighted_surface_energy_EV_PER_ANG2
weighted_work_function
xas
```

7.5.3 トラブルシューティング

getcif 実行時に起こりやすい問題と対処法を以下にまとめます。

- **検索結果が 0 件になる**

  検索条件が厳しすぎると、条件に合致する物質が存在せず結果が 0 件になることがあります。 `properties`

の条件を緩める、または --dry-run オプションで実際に送られる検索条件を確認した上で、条件を見直してください。

- **API キーが見つからない**

  API キーが取得できない場合、Materials Project への接続に失敗します。getcif はまず database セクションの api_key_file (デフォルト materials_project.key) で指定された .key ファイルを読み込みます。ファイルが存在する場合はそれが優先され、先頭のコメント行以外の行が使われます (空であってはいけません)。見つからない (または先頭の非コメント行が空の) 場合はキーの解決を MPRester に委譲し、環境変数 MP_API_KEY や pymatgen の設定ファイル ~/.config/.pmgrc.yaml の PMG_MAPI_KEY が参照されます。いずれかの方法で有効な API キーが設定されているか確認してください (チュートリアルの「API キーを取得する」を参照)。

- **fields check failed / unknown query key**

  fields に存在しない出力項目名を指定すると unknown field name に続いて fields check failed のエラーが発生します。同様に、 properties に未知の検索条件キーを指定すると unknown query key のエラーが発生します。本章の出力項目一覧および検索条件のキーワード表を参照し、項目名のスペルを確認してください。